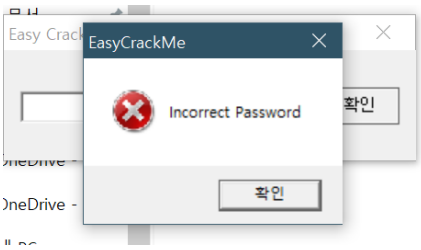
Easy Crack

해당 프로그램을 분석하기 위해 주요코드들이 어디있는지 찾기 위해 우선 실행을 해보았고, Incorrect Password 라는 문자열이 언급되는 부분을 중심으로 찾아보기 시작하면 될 것 같다.



아마 저 일치하는 password를 찾는 것이 문제인 것 같고, 코드 내에서 password를 인증하는 함수가 있을것이니 해당 인증연산을 역산하면 될 것이다.

Ida를 이용해서 해당 문자열이 언급되는 부분을 확인해보니

```
int __cdecl sub_401080(HWND hDlg)
{
  CHAR String[97]; // [esp+4h] [ebp-64h] BYREF
  __int16 v3; // [esp+65h] [ebp-3h]
  char v4; // [esp+67h] [ebp-1h]

  memset(String, 0, sizeof(String));
  v3 = 0;
  v4 = 0;
  GetDlgItemTextA(hDlg, 1000, String, 100);
  if ( String[1] != 'a' || strncmp(&String[2], Str2, 2u) || strcmp(&String[4], aR3versing) || String[0] != 'E' )
    return MessageBoxA(hDlg, aIncorrectPassw, Caption, 0x10u);
  MessageBoxA(hDlg, Text, Caption, 0x40u);
  return EndDialog(hDlg, 0);
}
```

생각보다 더 쉽게 끝낼 수 있을 것 같다.

Str2가 뭔지만 알면 되는데, 이는 5y라는 것을 정적분석을 통해 알수있다.

```
string[1] != 'a' ||
strncmp(&String[2], Str2, 2u) ||
strcmp(&String[4], aR3versing) ||
String[0] != 'E' )
```

오른쪽 조건을 만족하면 끝이므로 Str2가 뭔지만 알면 끝이다

password는 **Ea5yR3versing**

Easy Keygen

시리얼번호가 5B134977135E7D13일 때의 이름을 찾으라고 한다.

이전 문제와 같이 문자열을 먼저 보고 이를 언급하는 함수를 찾아가보면 된다.

```
int __cdecl main(int argc, const char **argv, const char **envp)
{
  signed int v3; // ebp
  int i; // esi
  char v6; // [esp+Ch] [ebp-130h]
  _BYTE v7[2]; // [esp+Dh] [ebp-12Fh] BYREF
  char v8[100]; // [esp+10h] [ebp-12Ch] BYREF
  char Buffer[197]; // [esp+74h] [ebp-C8h] BYREF
  __int16 v10; // [esp+139h] [ebp-3h]
  char v11; // [esp+13Bh] [ebp-1h]

  memset(v8, 0, sizeof(v8));
  memset(Buffer, 0, sizeof(Buffer));
  v10 = 0;
  v11 = 0;
  v6 = 16;
  qmemcpy(v7, " 0", sizeof(v7));
  sub_4011B9(aInputName);
  scanf("%s", v8);
  v3 = 0;
  for ( i = 0; v3 < (int)strlen(v8); ++i )
  {
    if ( i >= 3 )
      i = 0;
    sprintf(Buffer, "%s%02X", Buffer, v8[v3++] ^ v7[i - 1]);
  }
  memset(v8, 0, sizeof(v8));
  sub_4011B9(aInputSerial);
  scanf("%s", v8);
  if ( !strcmp(v8, Buffer) )
    sub_4011B9(aCorrect);
  else
    sub_4011B9(aWrong);
  return 0;
}
```

해당 함수다.

이름으로부터 시리얼넘버를 계산하는데, 단순히 v7(0x20, 0x30)을 번갈아 xor하는 것 같다.

```
serial = "5B134977135E7D13"

serials = [int(serial[2*i:2*i+2], 16) for i in range(len(serial)//2)]
print(serials)
print("".join(chr(serials[i]^[0x10,0x20,0x30][i%3]) for i in range(len(serials))))
```

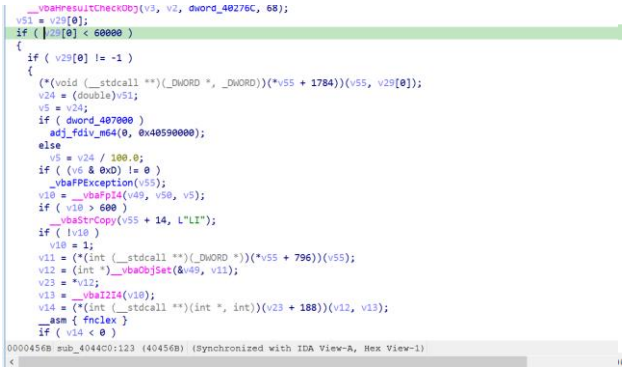
해당하는 name은 **K3yg3nm3**

Music Player

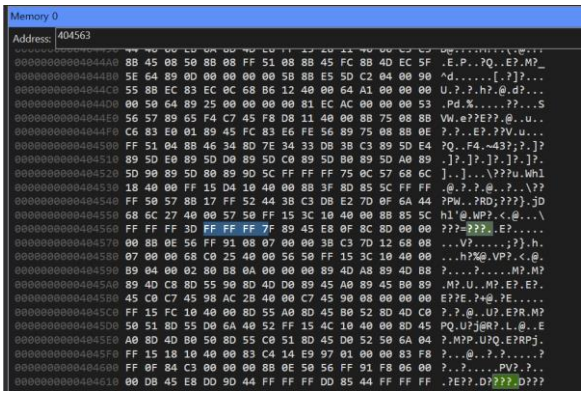
뮤직플레이어 프로그램에서 1분 듣기만 제공해주는데 해당 제약을 우회하는 방법을 찾는 것이 문제에서 요구하는 바이다.

결국 문자열이 어디서 접근되는지가 관건인데 여기서는 한글을 이용하기 때문에 IDA의 strings 기능에서 찾아주지를 않는다. 그래서 '1분'이라는 글자를 UTF-16LE 방식으로 적으면 '31 00 84 BD'이기 때문에 이렇게 검색을 했고 2BAC 위치에 있다는 것을 발견했다. 해당 주소의 xref을 찾아보면 될 것 같다. 참고로 IDA 내에서 Alt+A를 누르면 아래와 같이 인코딩 방식을 바꿀 수 있다.

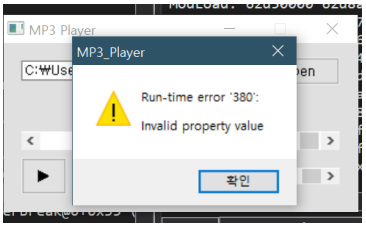
60초가 지나면 저 창이 뜨게 되는 것이니 대충 생각해보면 v29[0]이라는 변수가 시간을 나타내고 해당 값이 60000을 넘으면 창이 띄워지는 방식인 것 같다.



그래서 우선 0x7FFFFFFF 전까지는 계속 실행할 수 있도록 해놨다. windbg의 Memory 창에서 그냥 숫자를 입력하면 즉각적으로 바뀐다.



동적분석을 해보니 이 v29[0]에 들어있는 변수는 지나간 시간을 체크해주는 애인거같고, 1분을 기다려보면 오른쪽과 같은 에러메시지가 뜨는 것을 보니 올바른 접근 방식은 아닌 것으로 보인다.

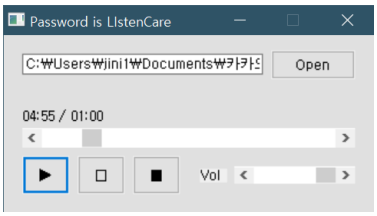


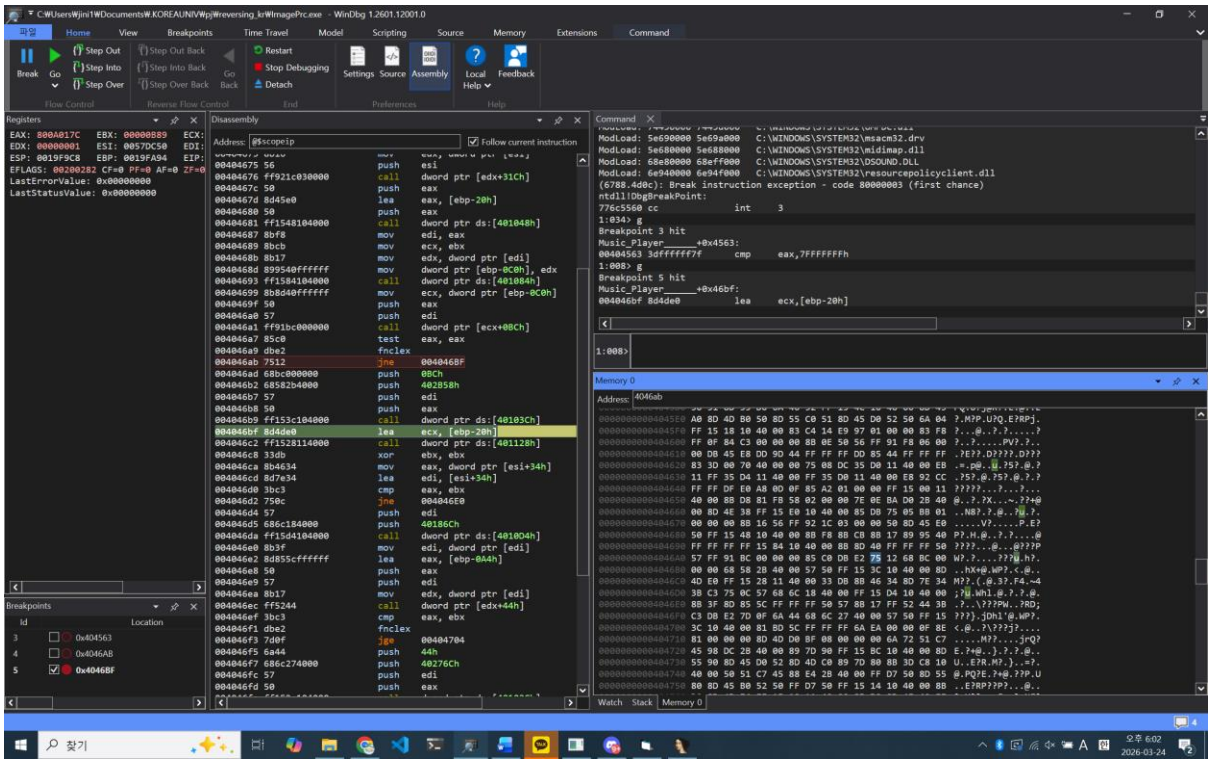
다시 브레이크포인트를 걸어서 시간이 60초를 넘었을 때 Runtime Error가 뜨게하는 코드는 아래 부분이였다.

```
text:0040469F                 push    eax
text:004046A0                 push    edi
text:004046A1                 call    dword ptr [ecx+0BCh]
text:004046A7                 test    eax, eax
text:004046A9                 fnclex
text:004046AB                 jge     short loc_4046BF
text:004046AD                 push    0BCh
text:004046B2                 push    offset aT_0     ; "伙rad晴zcsd■杯鎚獲扶噜牲
text:004046B7                 push    edi
text:004046B8                 push    eax
text:004046B9                 call    ds:__imp___vbaHresultCheckObj
text:004046BF
text:004046BF loc_4046BF:                             ; CODE XREF: music_playe
text:004046BF                 lea     ecx, [ebp+var_20]
text:004046C2                 call    ds:__imp___vbaFreeObj
text:004046C8                 xor     ebx, ebx
text:004046CA
```

```
138     __vbaStrCopy(a1a + 14, L"LI");
139   if ( !time_relevant )
140     time_relevant = 1;
141   v11 = (*(int (__stdcall **)(_DWORD *))(*a1a + 0x31C))(a1a);
142   v12 = (int *)__vbaObjSet(&v49, v11);
143   v23 = *v12;
144   v13 = __vbaI2I4(time_relevant);
145   v14 = (*(int (__stdcall **)(int *, int))(v23 + 0xBC))(v12, v13);
146   __asm { fnclex }
147   if ( v14 < 0 )
148     __vbaHresultCheckObj(v14, v12, aT_0, 0xBC);
149   __vbaFreeObj(&v49);
150 }
151 if ( !a1a[13] )
152   __vbaNew2(dword_40186C, a1a + 13);
153 v15 = a1a[13];
154 v16 = (*(int (__stdcall **)(int, _DWORD *))(*(_DWORD *)v15 + 68))(v15,
```

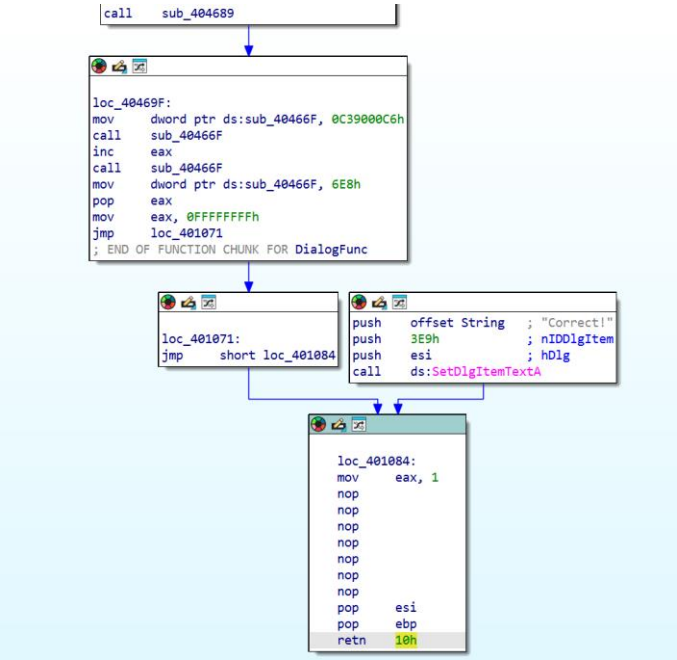
ai한테물어보니까 error code 380이 뜨게하는 것은 Max값을 넘어서 시간이 재생되었기 때문이라고 하고 스크롤바에서 그게 나오게되는것같다. 아무튼 저 예외처리를 하는 부분도 그냥 없애버리고 진행시켰는데, 그냥 노래만 재생됐었다. 근데 프로그램을 다시 확인해보니 이름이 바뀌어있었다.





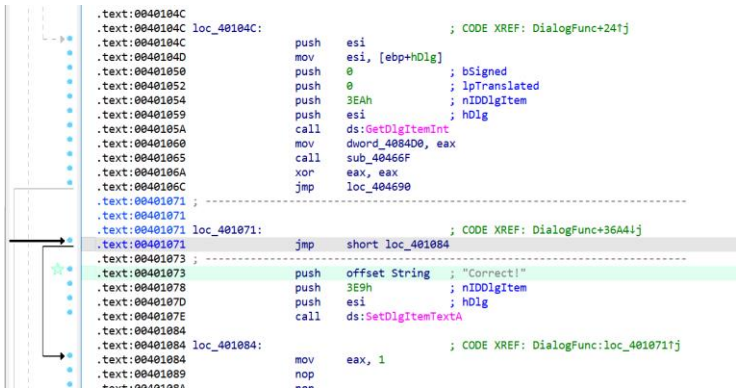
Replace

대충 문제 이름을 보면 어떤 부분을 패치하는 것 같다.



Correct라는 문자열이 나오기 위한 방법을 찾는 중에 Correct라는 결과를 내는 경로자체가 없다는 것을 알 수 있었다.

이와 같이 IDA의 그래프 기능을 이용하면 함수의 흐름이 어떻게 되는지 파악하는데 이점이 있다.



jmp를 단순히 없애면 Correct가 뜨긴 할텐데 그렇게하면 reversing.kr에 제출할 답 자체가 없다. 그러니 다른 방법을 찾아보도록 하겠다.

디스어셈블된 코드를 보면 입력값에 대해 특정한 연산을 가하는 것이 보이기는 하는데 어떤 부분을 어디부터 어디까지 패치해야 올바르게 패치를 한 것인지 감이 오질 않는다.

동적분석을 하면서 input이 어떻게 변하는지 확인해보면 input <- input + 2 + 601605c7 + 2 = input + 601605cb와 같이 변한다.

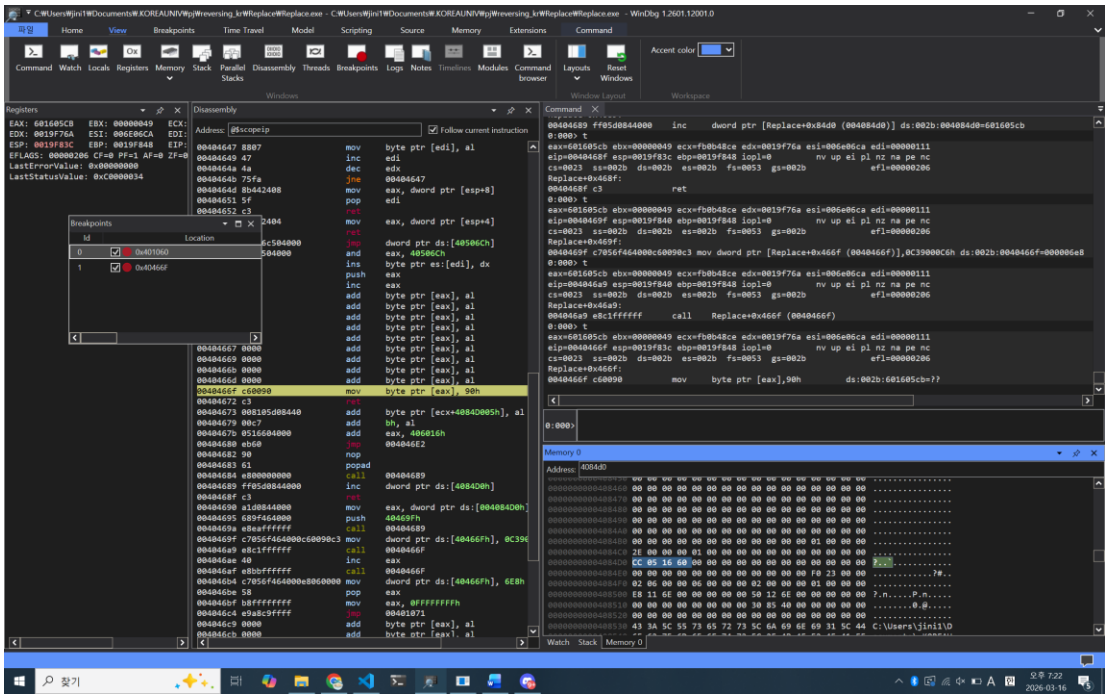
돌다보면 아래코드처럼 입력값 주소와 그 뒤에 있는 값을 0x90(nop)으로 바꾸는 코드가 있는데, 이 입력값이 잘못되면 권한침해로 프로세스가 끝난다. (eax = 입력값)

```
0040466d 0000                add     byte ptr [eax], al
0040466f c60090              mov     byte ptr [eax], 90h
00404672 c3                  ret
00404673 008105d08440        add     byte ptr [ecx+4084D005h], al
```

아마 jmp instruction이 2바이트이기 때문에 딱 그 위치에 집어넣으면 딱일 것 같다. 그러면 eax = 0x401071이 되게 하면 된다.

답은 2687109798

프로그램을 패치하는 것인가 했는데 이 프로그램 자체가 데이터를 수정하는 것이기 때문에 내가 했던 고민이 전혀 상관없이 아주 명확한 답이 있는 문제였다.



ImagePrc

```
  }
  if ( wParam == 0x64 )
  {
    GetObjectA(hbm, 24, pv);
    memset(&bmi, 0, 0x28u);
    bmi.bmiHeader.biHeight = cLines;
    bmi.bmiHeader.biWidth = v16;
    bmi.bmiHeader.biSize = 40;
    bmi.bmiHeader.biPlanes = 1;
    bmi.bmiHeader.biBitCount = 24;
    bmi.bmiHeader.biCompression = 0;
    GetDIBits(hdc, (HBITMAP)hbm, 0, cLines, 0, &bmi, 0);
    v8 = operator new(bmi.bmiHeader.biSizeImage);
    GetDIBits(hdc, (HBITMAP)hbm, 0, cLines, v8, &bmi, 0);
    ResourceA = FindResourceA(0, (LPCSTR)0x65, (LPCSTR)0x18);
    Resource = LoadResource(0, ResourceA);
    v11 = LockResource(Resource);
    v12 = 0;
    v13 = v8;
    v14 = v11 - (_BYTE *)v8;
    while ( *v13 == v13[v14] )
    {
      ++v12;
      ++v13;
      if ( v12 >= 0x15F90 )
      {
        sub_401500(v8);
        return 0;
      }
    }
    MessageBoxA(hWnd, Text, Caption, 0x30u);  // wrong
    sub_401500(v8);
    return 0;
  }
```

해당 함수에서 내가 한 싸인이랑 비교를 진행하는 것 같고 비교 대상이 되는 메모리를 그대로 복사해서 붙여넣으면 될 것 같다.

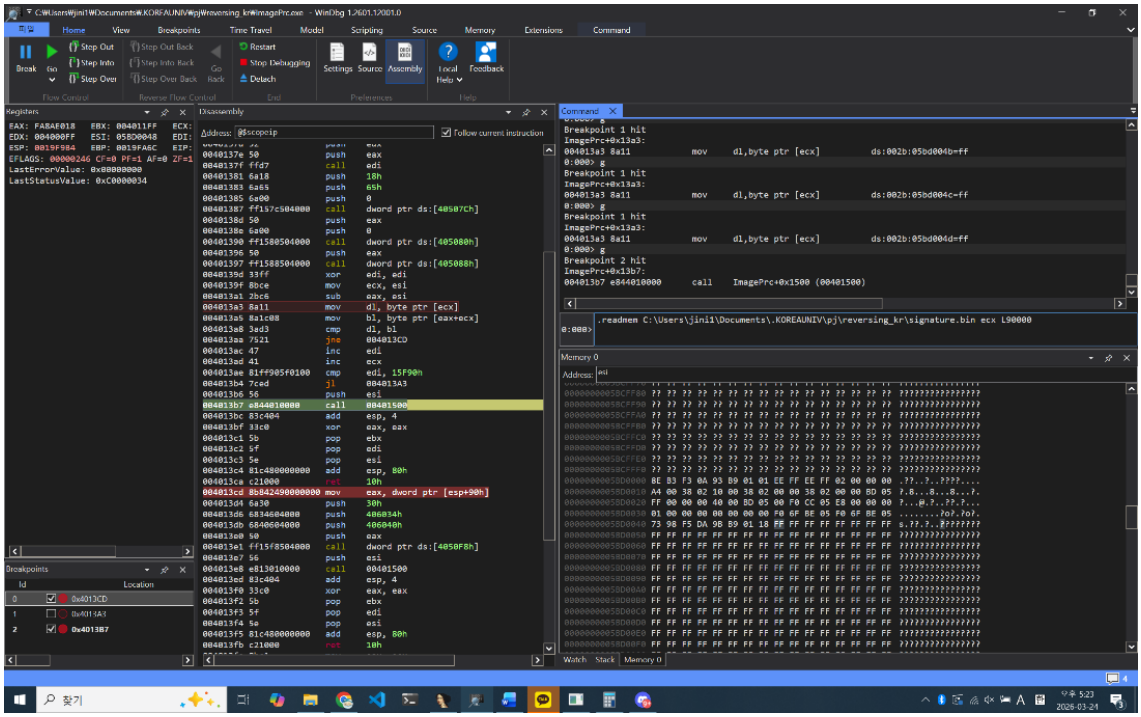
windbg에서

> .writemem [저장위치] [주소] [크기]

를 입력하여 싸인을 파일로 뽑아내고, 이 포맷이 bmp 이미지 파일과 동일하여 확인해보니 오르쪽과 같은 싸인이 나왔다.

같은 방식으로

> .readmem [저장위치] [주소] [크기]

을 입력하여 비교연산을 통과시킬 수 있다. 그래서 했는데 아무것도 변한게없다.

너무 짧아서 아니겠거니 했는데, 그냥 저 사진에 있는 내용이 정답이였다.